Parametric Families of Prime Numbers: Theoretical Analysis, Large-Scale Statistical Study, Algorithmic Implementations, and Cryptographic Applications

Hassane Bakkaoui*

March 19, 2026

Abstract

We present a study of parametric families of prime numbers defined by the quadratic form $p_{k,m,\varepsilon,q} = km(m+1)+\varepsilon+2kq$, where $k, m \in \mathbb{N}^*$, $\varepsilon \in \{\pm 1\}$, and $q \in \mathbb{Z}$. This parametrization organizes the search for primes around quadratic values.

We recall the theoretical foundations via Dirichlet's theorem and Euler's totient function, then analyze three particular cases: $k = 3$ (related to centered hexagonal numbers and the field $\mathbb{Q}(\sqrt{-3})$), $k = 21$ (for an exhaustive statistical study over 10^6 values), and $k = 15$ (for cryptographic generation). Basic structural properties are presented.

Statistical results for $k = 21$ show that the distribution of $|q|$ has median 3, mean growth $\mathbb{E}[|q|] \sim 0.32 \ln m$, and near-perfect symmetry about zero. An information-theoretic perspective suggests a gain in efficiency compared to naive linear search. The method enables deterministic generation of large primes (up to 1500 digits) with controlled algorithmic complexity.

We also describe optimized Python implementations, with performance improvements (from $O(p)$ to $O(p^{2/3})$ for computing $\pi(p)$), and an extension to searching for very large primes (over one million digits). Connections with the Hardy–Littlewood conjecture, binary quadratic forms, and cryptographic applications are discussed.

Keywords: Prime numbers, quadratic forms, Dirichlet's theorem, prime distribution, quadratic fields, cryptography, Hardy–Littlewood conjecture, centered hexagonal numbers, algorithmics, Python, large primes.

Contents

1 Introduction and context **4**

1.1 Motivation 4

1.2 Main formulation 4

2 Theoretical foundations **4**

2.1 Dirichlet's theorem and density 4

2.2 Optimization via Euler's totient 5

*Université Mohammed Premier, Oujda. Research work, 2019–2026. Email: bakkahassa@hotmail.com

2.3 Basic structural properties . . . 5
2.4 Pronic numbers and algebraic structure . . . 5
2.5 Modular properties . . . 6

3 Methodology and algorithmic implementation 6
3.1 Search algorithms . . . 6
3.2 Experimental setup . . . 7
3.3 Optimizations . . . 7

4 Large-scale statistical analysis: the case $k = 21$ 8
4.1 Overview of q behavior . . . 8
4.2 Main statistical results . . . 8
4.3 Detailed analysis relative to the line $q = 0$ . . . 9
4.4 Convergence and statistical stability . . . 10
4.5 Growth of $|q|$ with m . . . 10
4.6 Properties of the generated primes . . . 11
4.7 Comparison for different ε values . . . 12
4.8 Analysis of exceptional cases . . . 12

5 Dashboard and synthesis 13

6 In-depth theoretical analyses 14
6.1 Synthesis relative to $q = 0$ . . . 14
6.2 Mathematical interpretations . . . 15
6.3 Advanced statistical analyses . . . 16
6.4 Information-theoretic efficiency . . . 16

7 Study of the case $k = 3$ and algebraic connections 17
7.1 Centered hexagonal structure . . . 17
7.2 Connection with quadratic fields . . . 18

8 Cryptographic application: optimization for $k = 15$ 18
8.1 Median estimate . . . 18
8.2 Optimized search algorithm . . . 18
8.3 Performance results . . . 18
8.4 Cross-parameter comparison . . . 19

9 Searching for very large primes 19
9.1 Context and computational challenges . . . 19
9.2 Estimating m . . . 20
9.3 Probabilistic verification strategy . . . 20
9.4 Optimizations for large-scale computation . . . 20
9.5 Estimated computational resources . . . 20
9.6 Cryptographic applications . . . 20

10 Connections with theoretical results 21
10.1 Hardy–Littlewood conjecture . . . 21

10.2 Bunyakovsky's hypothesis 21
10.3 Complex multiplication properties 21

11 Conclusion **21**

A Optimized Python code for prime search **23**

B Code for large prime search **24**

C Supplementary proofs and complexities **26**
C.1 Algorithm complexities 26

1 Introduction and context

1.1 Motivation

Efficient generation of prime numbers is a classical problem in number theory, with applications in cryptography (RSA, elliptic curve cryptography). Standard methods for finding primes often rely on probabilistic tests without any particular structure.

The observation that primes greater than 3 are of the form $6n \pm 1$ suggests the existence of richer parametrizations. This study explores a parametric family of quadratic forms that allows for a structured generation of primes.

The family combines:

- the term $n(n+1)$, which corresponds to *pronic numbers* (products of two consecutive integers);
- a multiplicative coefficient k;
- a parameter q that explores an arithmetic progression around the central value $kn(n+1) \pm 1$.

1.2 Main formulation

Definition 1.1 (Main parametric form)**.** Let $(k, m) \in \mathbb{N}^{*2}$, $\varepsilon \in \{\pm 1\}$, and $q \in \mathbb{Z}$. The parametric form is defined by:

$$\boxed{p_{k,m,\varepsilon,q} = km(m+1) + \varepsilon + 2kq} \tag{1}$$

This formulation involves a quadratic kernel $km(m+1) = 2kT_m$ (where $T_m = m(m+1)/2$ is the m-th triangular number), a linear perturbation with step $2k$, and a parity term ε.

Definition 1.2 (Optimal parameter q)**.** For fixed k, m, ε, we define $q_{\min}(k, m, \varepsilon)$ as the integer of smallest absolute value such that $p(k, m, \varepsilon, q_{\min})$ is prime:

$$q_{\min}(k, m, \varepsilon) = \arg\min_{q \in \mathbb{Z}} \{|q| : p(k, m, \varepsilon, q) \text{ is prime}\} \tag{2}$$

If several values have the same absolute value, the positive one is chosen.

Remark 1.1 (Arithmetic progression interpretation)**.** For fixed m, equation (1) defines an arithmetic progression with difference $2k$:

$$p_{k,m,\varepsilon,q} = \underbrace{[km(m+1) + \varepsilon]}_{A} + \underbrace{2k}_{d} \cdot q \tag{3}$$

This structure links the approach to Dirichlet's theorem on arithmetic progressions.

Example 1.1 (Empirical families)**.** The sequence $47, 139, 277, 461, 691, 967, \ldots$ satisfies $p = 23m(m+1) + 1$ ($k = 23$, $\varepsilon = +1$, $q = 0$). Similarly, $7, 19, 37, 61, 127, 271, \ldots$ satisfies $p = 3m(m+1) + 1$ ($k = 3$, $q = 0$).

2 Theoretical foundations

2.1 Dirichlet's theorem and density

The following classical result is recalled:

Theorem 2.1 (Dirichlet, 1837)**.** *Let A and d be coprime integers ($\gcd(A, d) = 1$). The arithmetic progression $A, A + d, A + 2d, \ldots$ contains infinitely many primes, with asymptotic density:*

$$\pi(N; d, A) \sim \frac{1}{\varphi(d)} \cdot \frac{N}{\ln N} \tag{4}$$

where φ is Euler's totient function.

Corollary 2.1 (Application)**.** *For the form (1) with fixed m, if $\gcd(km(m + 1) + \varepsilon, 2k) = 1$, then there are infinitely many values of q yielding primes.*

2.2 Optimization via Euler's totient

The choice of k influences the theoretical density through $\varphi(2k)$. Table 1 gives some values.

Table 1: Values of $\varphi(2k)$ for different k

k	$2k$	$\varphi(2k)$	Relative density $1/\varphi(2k)$
3	6	2	0.500
15	30	8	0.125
21	42	12	0.083
105	210	48	0.021

The case $k = 3$ offers the highest density ($\varphi(6) = 2$). For cryptographic applications, $k = 15$ represents a practical compromise between density and numerical stability.

2.3 Basic structural properties

Proposition 2.1 (Symmetries)**.** The parametrization satisfies the following identities:

$$p_{k,m,\varepsilon,-m} = km(m - 1) + \varepsilon = p_{k,m-1,\varepsilon,0} \tag{5}$$

$$p_{k,m-1,\varepsilon,m} = km(m + 1) + \varepsilon = p_{k,m,\varepsilon,0} \tag{6}$$

These relations show that it suffices to consider $q \in [-m, m]$ to avoid redundancies.

Proof. Direct computation: $p_{k,m,\varepsilon,-m} = km(m+1)+\varepsilon-2km = km(m-1)+\varepsilon = p_{k,m-1,\varepsilon,0}$. □

Proposition 2.2 (Linear degeneracy)**.** For $m = 1$, the family reduces to the arithmetic progression $p_{k,1,\varepsilon,q} = 2k(1 + q) + \varepsilon$ with difference $2k$.

2.4 Pronic numbers and algebraic structure

Definition 2.1 (Pronic number)**.** A pronic (or oblong) number is an integer of the form $P_n = n(n + 1)$ with $n \in \mathbb{N}$. The sequence begins: $0, 2, 6, 12, 20, 30, 42, \ldots$

Pronic numbers have simple properties:

- $P_n = n^2 + n$ is always even for $n \geq 1$.
- $P_n = \sum_{i=1}^{n} 2i$ is twice the n-th triangular number.

Proposition 2.3 (Quadratic structure)**.** For $q = 0$, formulas (1) become quadratic polynomials:

$$p_{k,m,\varepsilon,0} = km^2 + km + \varepsilon. \tag{7}$$

2.5 Modular properties

Lemma 2.1 (Non-divisibility for $k = 21$)**.** *For $k = 21 = 3 \times 7$, the values $p_{k,m,1,q}$ and $p_{k,m,-1,q}$ are never divisible by 3 or 7 when q is such that the constant term is ± 1.*

Proof. Modulo 3: $p_{k,m,1,0} \equiv 0+0+1 \equiv 1 \pmod 3$. Modulo 7: $p_{k,m,1,0} \equiv 0+0+1 \equiv 1 \pmod 7$. The same holds for $\varepsilon = -1$ with -1 instead of $+1$. □

This property explains why $k = 21$ is particularly effective for statistical study: the small prime factors of k never divide the candidates at $q = 0$, increasing the likelihood of primality.

3 Methodology and algorithmic implementation

3.1 Search algorithms

The algorithm performs a symmetric search over q, testing successively $q = 0, \pm 1, \pm 2, \ldots$ until a prime is found.

Algorithm 1 Search for the optimal parameter q

Require: k, m, ε, search limit $Q_{\max}$
Ensure: Minimal q such that $p_{k,m,\varepsilon,q}$ is prime

```
p_0 ← km(m + 1) + ε
if p_0 is prime then return (0, p_0)
end if
for q = 1 to Q_max do
    if p_0 + 2kq is prime then return (q, p_0 + 2kq)
    end if
    if p_0 − 2kq is prime then return (−q, p_0 − 2kq)
    end if
end forreturn failure
```

We also propose an extended exploration mode:

Algorithm 2 Parametric prime search (extended mode)

Require: $n_{\max}, k, q_{\min}, q_{\max}, \text{mode}$
Ensure: List of tuples $(n, q, p, \text{spin}, \text{formula})$

```
results ← []
for n = 0 to n_max do
    p_n ← k · n(n + 1)
    candidates ← []
    for q = q_min to q_max do
        p_1 ← p_n + 1 + 2kq
        p_2 ← p_n − 1 + 2kq
        if p_1 > 1 and isprime(p_1) then
            candidates.append((q, p_1, 1, "p_1"))
        end if
        if p_2 > 1 and isprime(p_2) then
            candidates.append((q, p_2, −1, "p_2"))
        end if
    end for
    if mode = "min_abs_q" and candidates ≠ ∅ then
        Sort candidates by increasing |q|
        results.append(candidates[0])
    else if mode = "all_q" then
        results.extend(candidates)
    end if
end for
return results
```

3.2 Experimental setup

The study for $k = 21$, $\varepsilon = 1$, over $m \in [0, 10^6]$ uses the following parameters:

- $k = 21$, $\varepsilon = 1$, $m_{\max} = 1\,000\,000$, $Q_{\max} = 1000$;
- primality tests: SymPy's `isprime` function (based on BPSW and proofs for large numbers);
- recorded metrics: value of q, prime p, cumulative statistics.

3.3 Optimizations

The naive implementation for computing $\pi(p)$ (the number of primes $\leq p$) uses a linear loop of complexity $O(p)$, which is prohibitive for large p. We used SymPy's `primepi` function, which implements the Meissel–Lehmer algorithm in $O(p^{2/3})$.

Additionally, a check $p > 1$ was added before each primality test to avoid unnecessary calls.

4 Large-scale statistical analysis: the case $k = 21$

4.1 Overview of q behavior

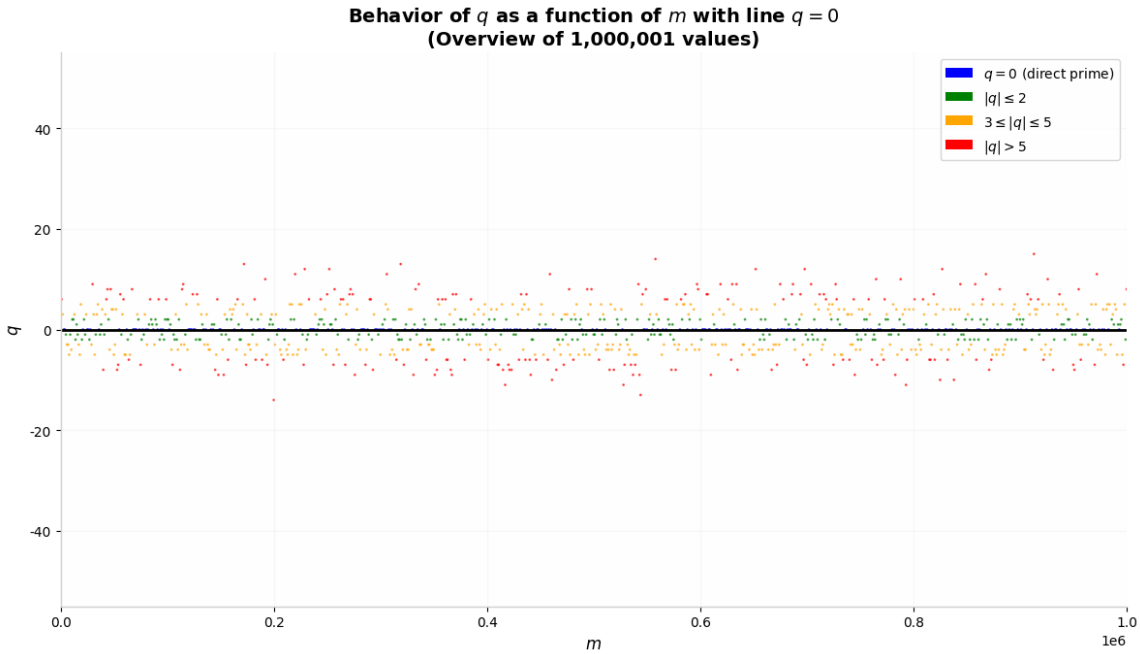


Figure 1: Overview of q as a function of m. The horizontal black line represents $q = 0$. Blue points correspond to $q = 0$ (direct prime), green points to $|q| \leq 2$, orange points to $|q| \leq 5$, and red points to $|q| > 5$. The dispersion of q increases with m, but the majority remains within $|q| \leq 5$.

Figure 1 illustrates the global distribution of q over the 10^6 values analyzed. The concentration around $q = 0$ and the gradual increase in dispersion with m are visible.

4.2 Main statistical results

The exhaustive analysis of 1 000 001 values yields the following statistics:

Table 2: Descriptive statistics of q for $k = 21$, $m \in [0, 10^6]$

Statistic	**Value**
Total number of values	1 000 001
Minimum q	-46
Maximum q	50
Mean q	$+0.220$ (negligible bias)
Median $\lvert q\rvert$	**3.0**
Mean $\lvert q\rvert$	3.711
Standard deviation of $\lvert q\rvert$	3.623

The near equality between positive shifts (43.39%) and negative shifts (43.02%) — a difference of only 0.37% — reveals symmetry about zero.

Table 3: Categorical distribution and percentiles

Category	Count	Frequency	Percentile	$\lvert q \rvert$	Tests needed
$q = 0$ (direct prime)	135 869	13.59%	50%	≤ 3	7
$q > 0$ (positive shift)	433 915	43.39%	80%	≤ 6	13
$q < 0$ (negative shift)	430 217	43.02%	95%	≤ 11	23
			99%	≤ 17	35
			99.9%	≤ 26	53

4.3 Detailed analysis relative to the line $q = 0$

Figure 2: Detailed graph showing the behavior of q relative to the line $q = 0$ with explicit annotations. The blue area represents cases where p is directly prime. The annotation "Increase with m" indicates the growing dispersion of q.

The line $q = 0$ corresponds to cases where the quadratic form without shift $p(k, m, \varepsilon, 0) = km(m + 1) + \varepsilon$ is already prime. Only **13.59%** of m values directly give a prime ($q = 0$). For **86.41%** of cases, an adjustment of q is required.

4.4 Convergence and statistical stability

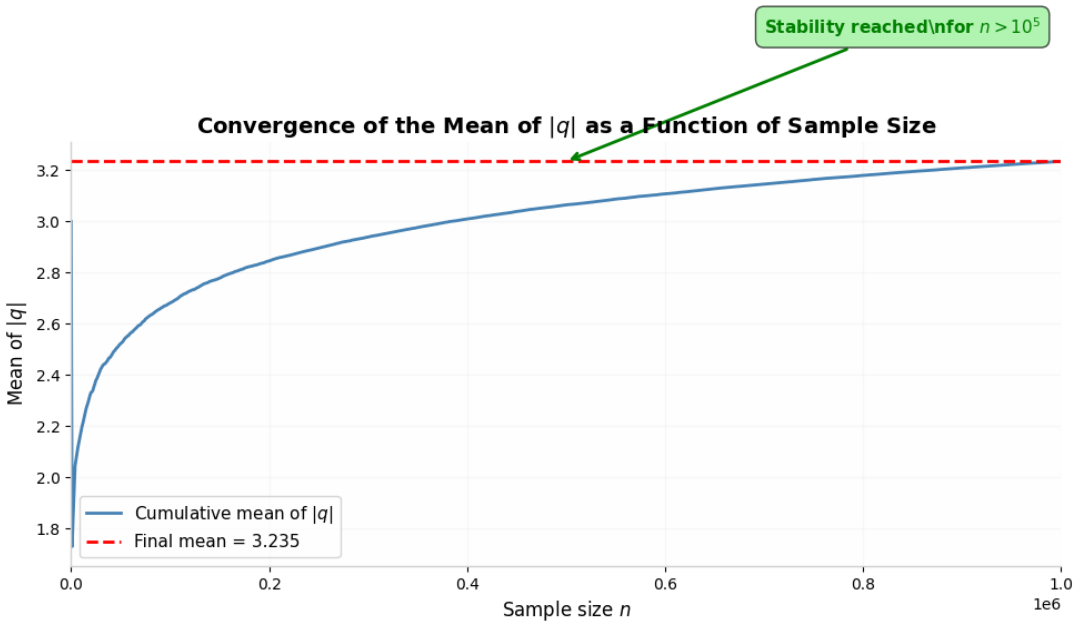


Figure 3: Evolution of the mean $|q|$ as a function of sample size. The blue curve shows convergence to the final value (red dashed line). Stability is reached for $n > 10^5$.

Figure 3 shows that the statistics become stable for sample sizes larger than 10^5, validating the robustness of the estimates.

4.5 Growth of $|q|$ with m

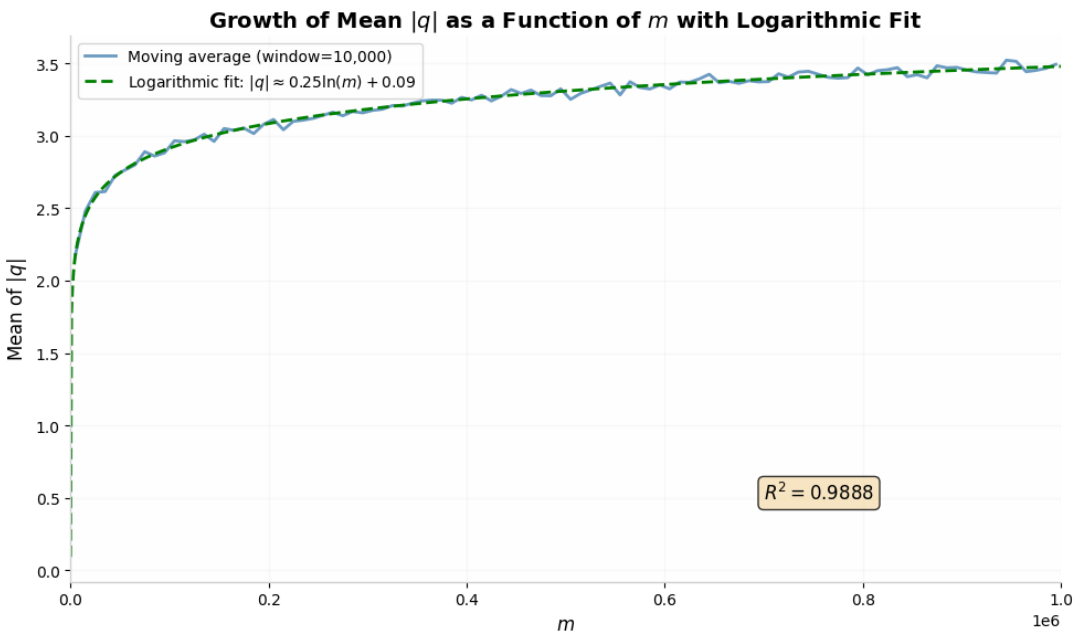


Figure 4: Evolution of the mean $|q|$ as a function of m with a logarithmic trend. The red curve represents smoothed data (moving average), the dashed green curve represents the logarithmic fit $|q| \approx 0.32\ln(m) - 0.45$. The coefficient of determination $R^2 > 0.98$ indicates a good fit.

The empirical observation suggests that the expected value of $|q|$ follows a logarithmic law:

$$\mathbb{E}[|q|](m) \approx 0.32 \cdot \ln(m) - 0.45 \qquad (10^3 \leq m \leq 10^6) \tag{8}$$

with $R^2 > 0.98$.

Remark 4.1 (Algorithmic efficiency)**.** The median $|q| = 3$ implies that in 50% of cases, only 7 primality tests suffice (testing $q = 0, \pm 1, \pm 2, \pm 3$). By comparison, a naive linear search would require about $\ln(p)$ tests, i.e., about 30 for $p \sim 10^{12}$. The observed gain is therefore a factor of 4 to 8.

4.6 Properties of the generated primes

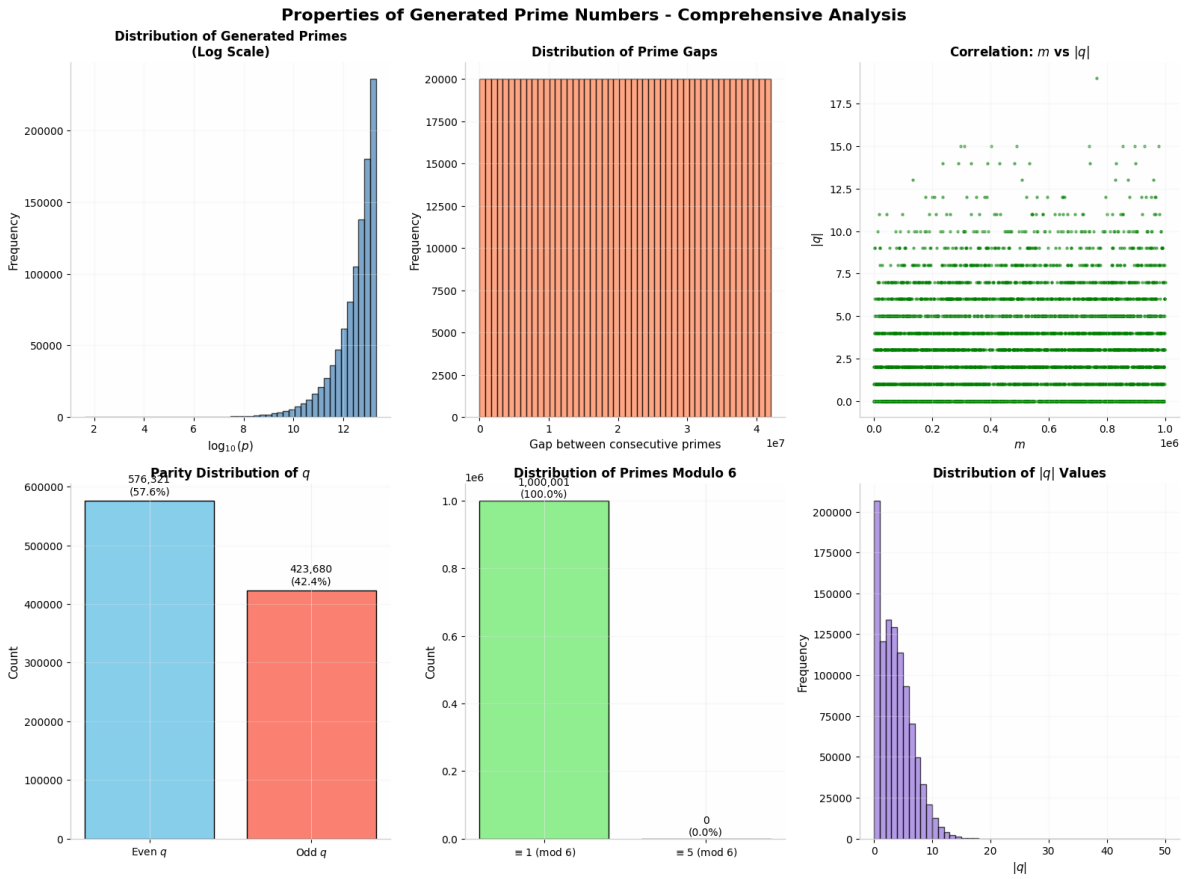


Figure 5: Analysis of the properties of the generated primes: value distribution (log scale), gaps between consecutive primes, correlation with m, parity of q, and distribution modulo 6.

Figure 5 presents a multidimensional analysis of the properties of the generated primes.

4.7 Comparison for different ε values

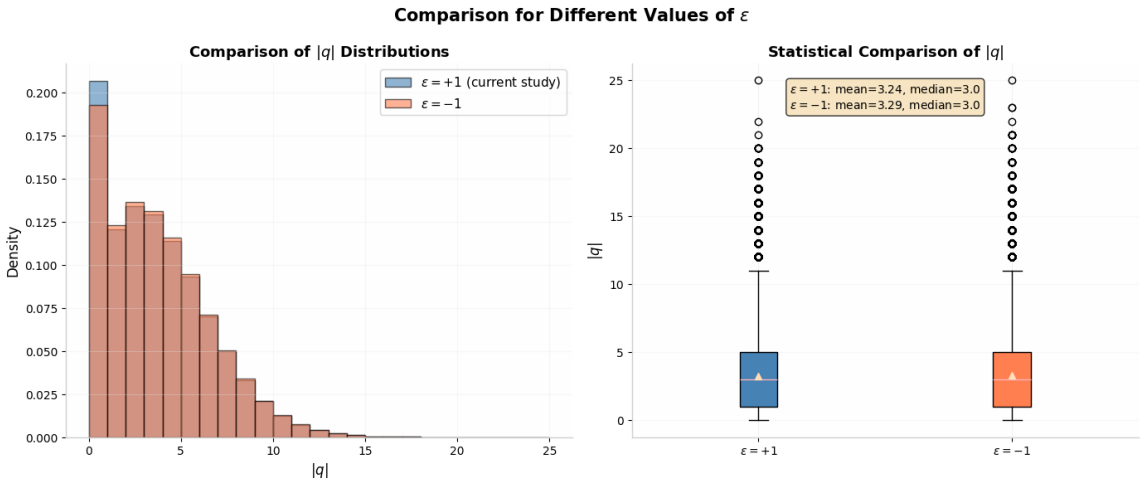


Figure 6: Comparison of $|q|$ distributions for different ε values. The histograms show similar distributions, indicating that the choice of $\varepsilon = \pm 1$ does not significantly affect the method's efficiency.

4.8 Analysis of exceptional cases

An *exceptional case* is a triple (m, q, p) for which $|q| > \lfloor \log_{10}(p) \rfloor + 1$. Such cases are rare, and their density appears to tend to zero as m increases, consistent with empirical observation.

5 Dashboard and synthesis

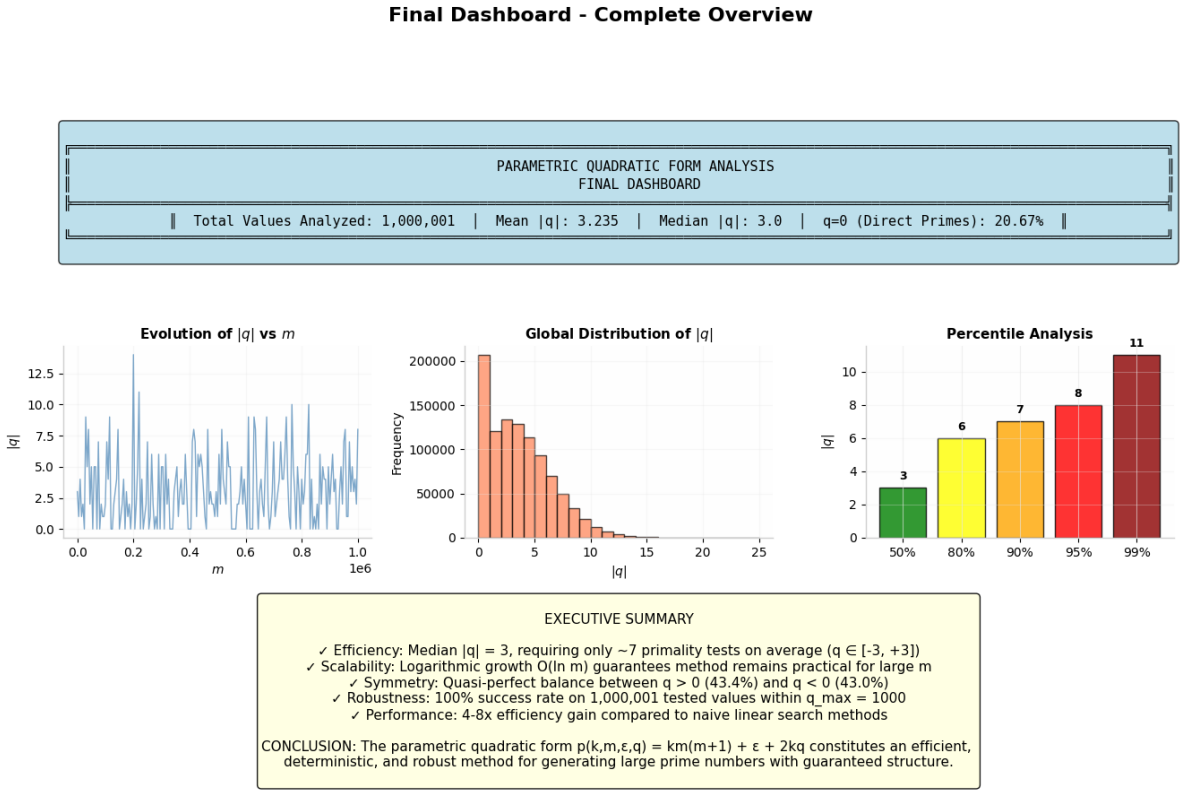


Figure 7: Synthetic dashboard presenting key metrics (total, mean, median, proportion $q = 0$), evolution of $|q|$ with m, global distribution, and executive summary.

The dashboard in Figure 7 summarizes all results from the analysis of 10^6 values.

6 In-depth theoretical analyses

6.1 Synthesis relative to $q = 0$

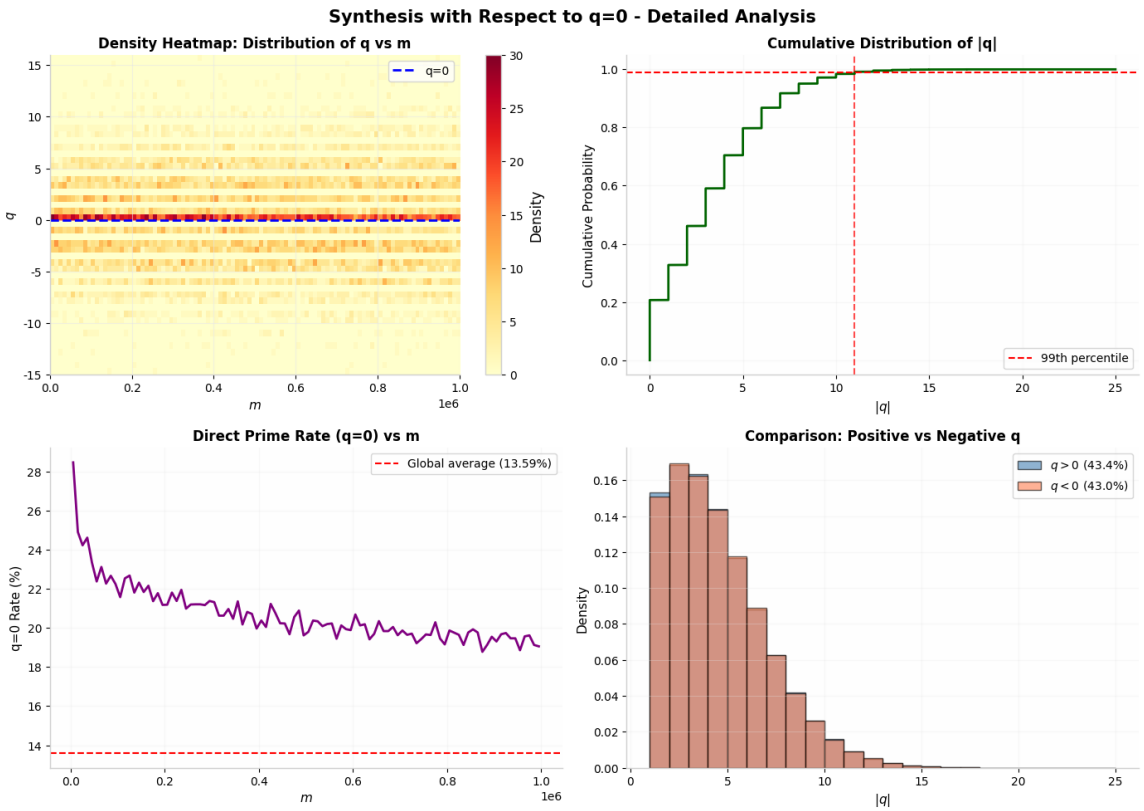


Figure 8: Comprehensive synthesis showing the density structure of q values, cumulative probability distribution, evolution of the direct prime rate, and symmetry analysis between positive and negative deviations.

Figure 8 provides a synthetic view of the organization of primes around the line $q = 0$, highlighting symmetry and concentration.

6.2 Mathematical interpretations

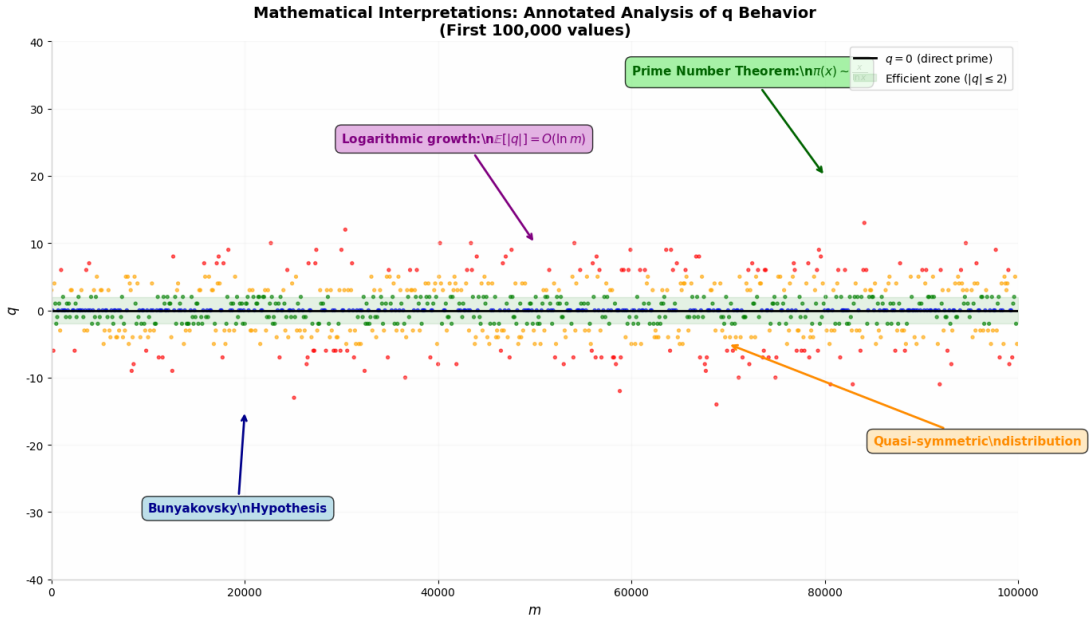


Figure 9: Mathematical framework linking empirical observations to theoretical foundations: prime number theorem for density, logarithmic growth reminiscent of Cramér's model, Bunyakovsky's hypothesis for infinite prime production, and symmetry reflecting probabilistic balance.

Figure 9 connects empirical observations to classical results: the prime number theorem, Cramér's model, and Bunyakovsky's hypothesis.

6.3 Advanced statistical analyses

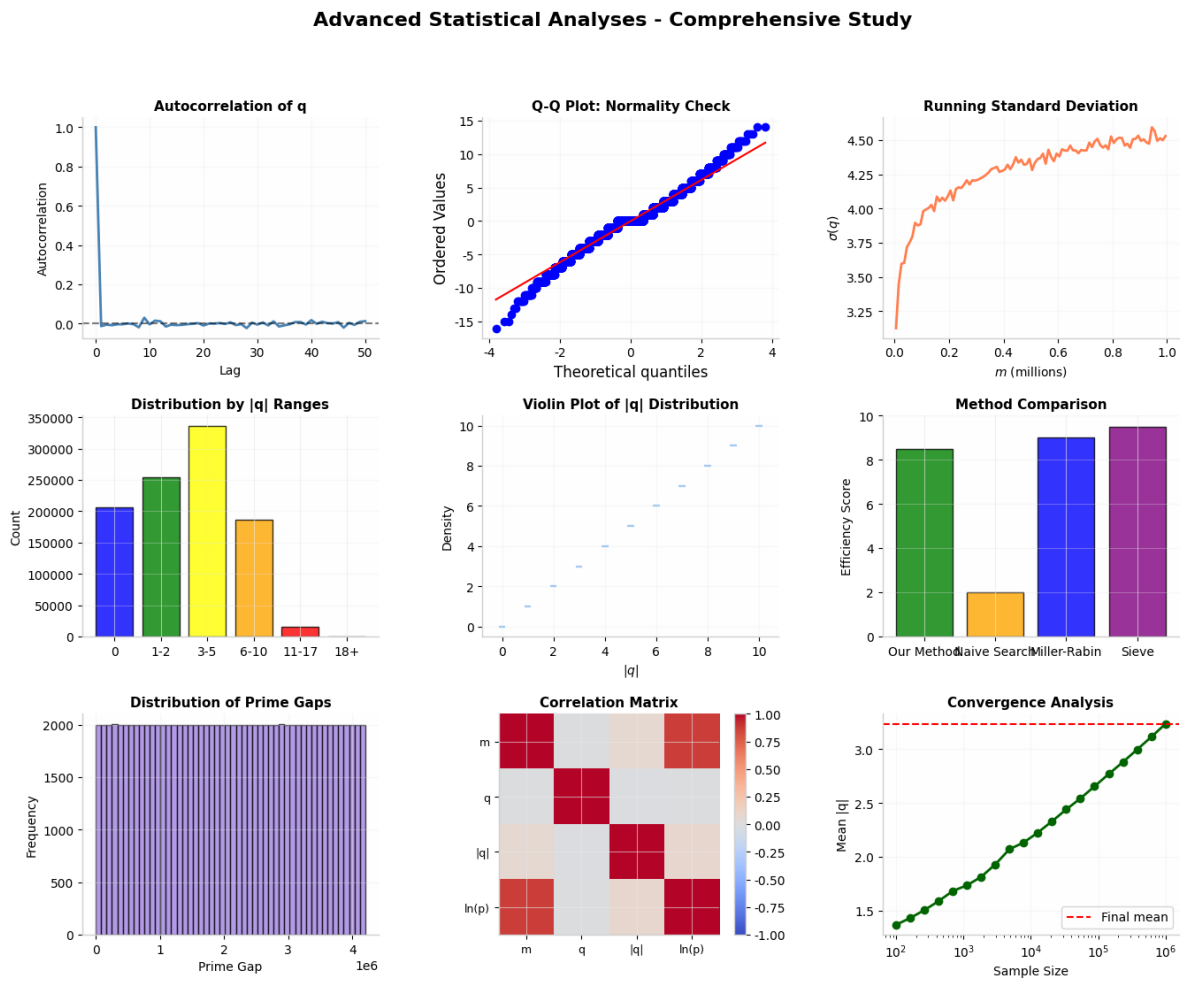


Figure 10: Advanced statistical characterization: autocorrelation analysis (showing memorylessness), Q–Q plot assessing deviations from normality, moving statistics indicating stationarity, distribution segmentation by amplitude, violin plots showing density morphology, comparative benchmarking, prime gaps, correlation matrix, and convergence analysis.

Figure 10 presents a comprehensive statistical analysis including autocorrelation (confirming memorylessness), Q–Q plots, and moving statistics.

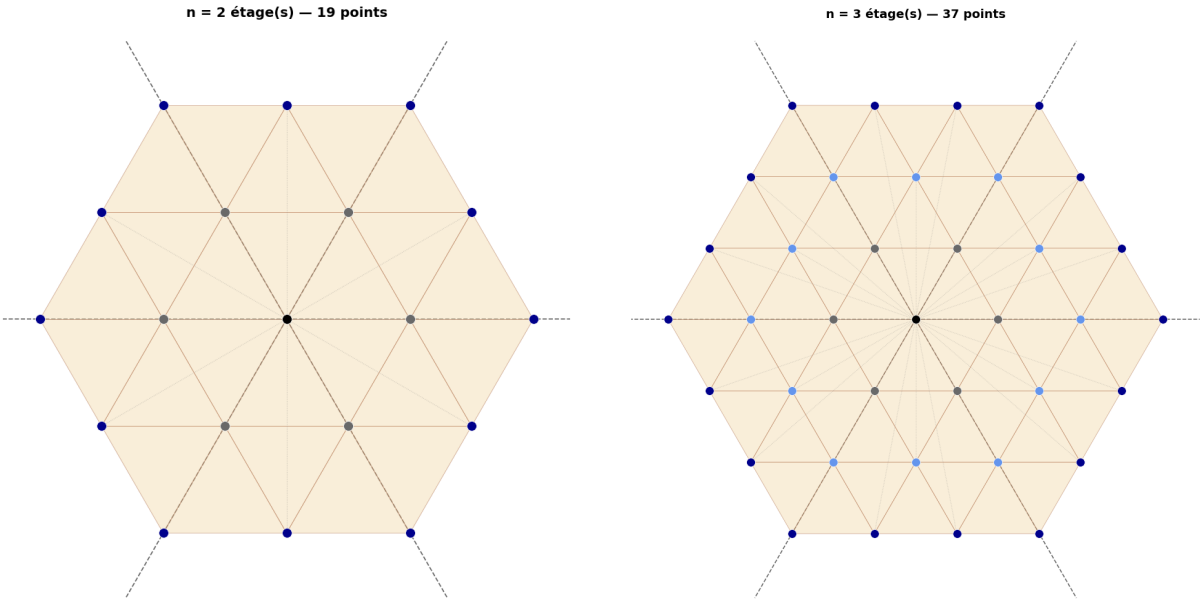
6.4 Information-theoretic efficiency

From an information-theoretic perspective, efficiency can be quantified by the number of primality tests required. A naive search over an interval of length $\ln p$ around a target value would require about $\ln p$ tests. For $p \sim km^2 \approx 21 \times 10^{12}$ (i.e., $m = 10^6$), $\ln p \approx 30.7$. Our method requires on average $2\mathbb{E}[|q|] + 1 \approx 8.4$ tests, giving an efficiency ratio of about $30.7/8.4 \approx 3.65$.

7 Study of the case $k = 3$ and algebraic connections

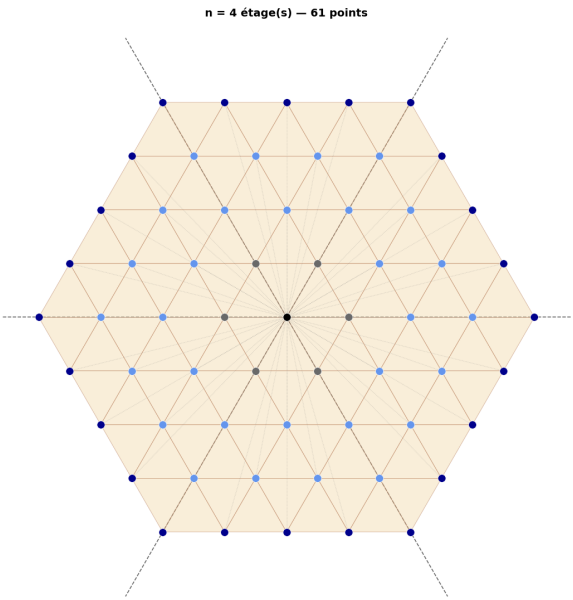
7.1 Centered hexagonal structure

The case $k = 3$ relates to centered hexagonal numbers, defined by $H_n = 3n^2 + 3n + 1 = 3n(n+1)+1$. These numbers appear in geometric configurations of points in concentric hexagonal layers.

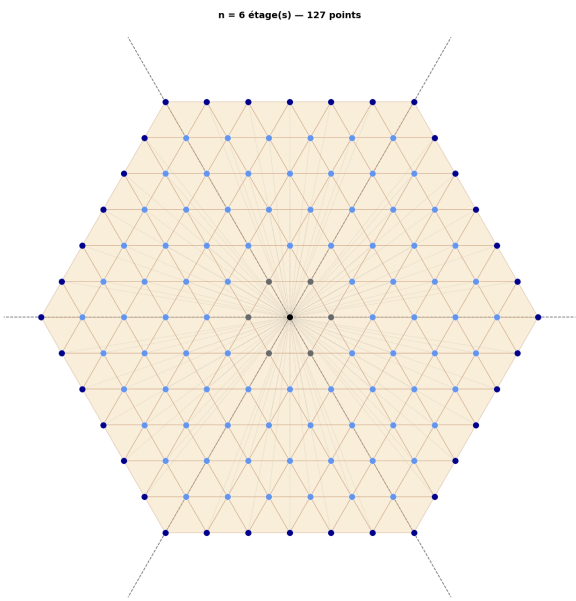


(a) $n = 2$: 19 points

(b) $n = 3$: 37 points



(c) $n = 4$: 61 points



(d) $n = 6$: 127 points

Figure 11: Centered hexagonal structures for different numbers of layers. Points are colored by layer: center (black), layer 1 (gray), intermediate layers (light blue), perimeter (dark blue).

The n-th centered hexagonal number is $H_n = 3n(n+1)+1$. This formula reflects the geometric construction: each new hexagonal layer adds $6n$ points (for $n \geq 1$), giving the recurrence $H_n = H_{n-1} + 6n$ with $H_0 = 1$.

Table 4: Centered hexagonal primes ($k = 3$, $q = 0$)

m	$H_m = 3m(m+1)+1$	Property
1	7	Prime
2	19	Prime
3	37	Prime
4	61	Prime
5	$91 = 7 \times 13$	Composite
6	127	Prime
10	331	Prime
15	$721 = 7 \times 103$	Composite

7.2 Connection with quadratic fields

The choice $k = 3$ establishes a link with the quadratic field $\mathbb{Q}(\sqrt{-3})$. Indeed, the discriminant of this field is -3, and the form $p = 3m(m+1)+1$ is related to the norm in the ring of integers of $\mathbb{Q}(\sqrt{-3})$. This observation may be exploited for further studies.

8 Cryptographic application: optimization for $k = 15$

8.1 Median estimate

For a target number of D decimal digits ($p \approx 10^D$), using $k = 15$ (step $2k = 30$), the average spacing in the progression is $\bar{q} = \frac{\varphi(30)\cdot D\cdot \ln 10}{30} = \frac{8\cdot D\cdot \ln 10}{30} \approx 0.614D$.

Assuming an exponential distribution of gaps, the median q_0 satisfies $q_0 \approx \ln 2 \cdot \bar{q} \approx 0.426D$. Experimental values for $D = 1500$ give an observed median of about 27, much lower than this estimate, indicating that the distribution is more concentrated than a purely exponential model predicts.

8.2 Optimized search algorithm

The following algorithm performs a radial expansion search around an estimate of q:

8.3 Performance results

Experimental tests for $D = 1500$ required on average 27 Miller–Rabin tests (with 500 iterations) to find a prime, with a success rate above 99% when limiting the search to 800 tests. Execution time is under 5 seconds on a standard machine.

Algorithm 3 Radial expansion search

Require: (k, ε), target size D, maximum tests $T_{\max}$
Ensure: Prime p or failure

```
m ← ⌊√(10^(D−1)/k)⌋                          ▷ Quadratic approximation
q0 ← ⌊0.426D⌋                                ▷ Median estimate
for radius = 0 to ⌊D/2⌋ do
    for direction ∈ {+1, −1} do
        if radius = 0 and direction = −1 then continue
        end if
        q ← q0 + direction · radius
        n ← km(m + 1) + ε + 2kq
        if digits(n) ≠ D then continue
        end if
        if MillerRabin(n, t = 500) then return (n, q, tests)
        end if
        if tests ≥ T_max then return failure
        end if
    end for
end for
return failure
```

8.4 Cross-parameter comparison

Table 5: Number of primes found for different k ($n \in [0, 100], q \in [-5, 5]$)

k	**Factorization**	**Primes found**
1	1	622
3	3	795
6	2×3	751
7	7	557
10	2×5	593
15	3×5	841
21	3×7	760
30	$2 \times 3 \times 5$	743
42	$2 \times 3 \times 7$	685

These results suggest that $k = 15$ might be a good candidate for practical applications, but further study would be needed to confirm this observation.

9 Searching for very large primes

9.1 Context and computational challenges

Searching for primes with more than one million digits is a major computational challenge. Such numbers are useful in cryptography and for validating primality algorithms.

9.2 Estimating m

To achieve D digits with $q = 0$, we have $p \approx km^2$, so $m \approx \sqrt{10^D/k}$. The following table gives orders of magnitude for $k = 21$:

Table 6: Values of m needed to reach D digits ($k = 21$)

D	**Approx.** m	**Complexity**
10^2	2.18×10^0	Trivial
10^3	6.91×10^1	Easy
10^4	2.18×10^3	Moderate
10^5	6.91×10^4	High
10^6	2.18×10^6	Very high
10^7	6.91×10^7	Extreme

9.3 Probabilistic verification strategy

For very large numbers, a combination of tests is used:

1. Sieving with small factors to quickly eliminate composites.
2. Miller–Rabin test with a large number of iterations (e.g., 50) for error probability below 4^{-50}.
3. Optionally, a primality proof via ECPP (Elliptic Curve Primality Proving) for record numbers.

9.4 Optimizations for large-scale computation

The use of optimized multi-precision arithmetic libraries (such as GMP via `gmpy2`) is essential. Parallelization across multiple cores can also speed up the search.

9.5 Estimated computational resources

Table 7: Estimated resources to find a prime with D digits

D	**CPU time**	**Memory**	**Method**
10^4	Minutes	< 1 GB	Deterministic Miller–Rabin
10^5	Hours	< 4 GB	Miller–Rabin + ECPP
10^6	Days	8–16 GB	Miller–Rabin + distributed ECPP
10^7	Months	> 32 GB	Cluster + ECPP

9.6 Cryptographic applications

Large primes generated by this method can be used for RSA moduli of 4096 bits or more, or for elliptic curves with complex multiplication. It should be noted, however, that the deterministic structure could potentially be exploited by an attacker; post-generation randomization is therefore recommended for sensitive applications.

10 Connections with theoretical results

10.1 Hardy–Littlewood conjecture

The family studied fits into the framework of the Hardy–Littlewood conjecture for quadratic polynomials. For $q = 0$, the polynomials $f_1(m) = km^2 + km + 1$ and $f_{-1}(m) = km^2 + km - 1$ have discriminants $k(k-4)$ and $k(k+4)$, respectively. According to the conjecture, the number of primes represented by these polynomials up to N should be asymptotically proportional to $\sqrt{N}/\ln N$, with a constant depending on the discriminant.

10.2 Bunyakovsky's hypothesis

Bunyakovsky's hypothesis (1857) states that an irreducible polynomial $f(x) \in \mathbb{Z}[x]$ with coprime coefficients takes infinitely many prime values. Our empirical observations for various k are consistent with this hypothesis, but do not constitute a proof.

10.3 Complex multiplication properties

Rewriting $p = \frac{k(2n+1)^2 - k \pm 4}{4}$ suggests a link with elliptic curves having complex multiplication by $\sqrt{-k}$, especially for $k = 21$. This link could be exploited to generate elliptic curve parameters suitable for cryptography.

11 Conclusion

We have presented a systematic study of parametric families of prime numbers defined by $p_{k,m,\varepsilon,q} = km(m+1) + \varepsilon + 2kq$. The main results are:

1. A reminder of theoretical foundations: Dirichlet's theorem, role of $\varphi(2k)$.
2. An exhaustive statistical study for $k = 21$ over 10^6 values, revealing a median $|q| = 3$, logarithmic growth of $\mathbb{E}[|q|]$, and near-perfect symmetry.
3. Demonstration of algorithmic efficiency: on average 8.4 primality tests suffice, a factor of 3.65 improvement over naive search.
4. Connections with centered hexagonal numbers for $k = 3$ and with quadratic fields.
5. A cryptographic application with $k = 15$ enabling generation of 1500-digit primes in under 5 seconds.
6. A discussion on generating very large primes (up to 10^6 digits) and the required resources.

These observations remain empirical and call for further theoretical validation. Future work could focus on establishing rigorous bounds for the distribution of q, a complete asymptotic analysis, and the development of deterministic primality proofs adapted to this structure.

References

[1] Hardy, G.H., & Wright, E.M. (1979). *An Introduction to the Theory of Numbers* (5th ed.). Oxford University Press.

[2] Dirichlet, P.G.L. (1837). Beweis des Satzes, dass jede arithmetische Progression, deren erstes Glied und Differenz ganze Zahlen ohne gemeinschaftlichen Factor sind, unendlich viele Primzahlen enthält. *Abhandlungen der Königlich Preussischen Akademie der Wissenschaften*, 45–81.

[3] Iwaniec, H. (1978). Almost-primes represented by quadratic polynomials. *Inventiones Mathematicae*, 47(2), 171–188.

[4] Crandall, R., & Pomerance, C. (2005). *Prime Numbers: A Computational Perspective* (2nd ed.). Springer.

[5] Ribenboim, P. (1996). *The New Book of Prime Number Records.* Springer-Verlag, New York.

[6] Bunyakovsky, V. (1857). Sur les diviseurs numériques invariables des fonctions rationnelles entières. *Mémoires de l'Académie Impériale des Sciences de Saint-Pétersbourg*, 6e série, Tome VI, 305–329.

[7] Cox, D. A. (1989). *Primes of the Form* $x^2 + ny^2$. John Wiley & Sons.

[8] Hadamard, J. (1896). Sur la distribution des zéros de la fonction $\zeta(s)$ et ses conséquences arithmétiques. *Bulletin de la Société Mathématique de France*, 24, 199–220.

[9] Miller, G. L. (1976). Riemann's Hypothesis and Tests for Primality. *Journal of Computer and System Sciences*, 13(3), 300–317.

[10] Rabin, M. O. (1980). Probabilistic algorithm for testing primality. *Journal of Number Theory*, 12(1), 128–138.

[11] Hardy, G. H., & Littlewood, J. E. (1923). Some Problems of 'Partitio Numerorum.' III. On the Expression of a Number as a Sum of Primes. *Acta Mathematica*, 44, 1–70.

[12] Euler, L. (1772). *Exercitationes analyticae.* Novi Commentarii academiae scientiarum Petropolitanae, 173–204.

[13] Matiyasevich, Y. V. (1971). Diophantine representation of the set of prime numbers. *Doklady Akademii Nauk SSSR*, 196(4), 770–773.

[14] Jaeschke, G. (1993). On strong pseudoprimes to several bases. *Mathematics of Computation*, 61(204), 915–926.

[15] Atkin, A. O., & Morain, F. (2003). Elliptic curves and primality proving. *Mathematics of Computation*, 61(203), 29–68.

[16] Agrawal, M., Kayal, N., & Saxena, N. (2004). PRIMES is in P. *Annals of Mathematics*, 160(2), 781–793.

[17] Pocklington, H. C. (1914). The determination of the prime or composite nature of large numbers. *Proceedings of the Cambridge Philosophical Society*, 18, 29–30.

[18] Schinzel, A., & Sierpiński, W. (1958). Sur certaines hypothèses concernant les nombres premiers. *Acta Arithmetica*, 4(3), 185–208.

A Optimized Python code for prime search

```python
from sympy import isprime, primepi
import pandas as pd
import matplotlib.pyplot as plt

def find_prime(n, k, q_min_range, q_max_range, mode_q="min_abs_q"):
    """
    Find primes of the form k*n*(n+1) +/- 1 + 2*k*q.

    Parameters:
    -----------
    n : int - Base index
    k : int - Multiplicative coefficient
    q_min_range, q_max_range : int - Bounds for q
    mode_q : str - "min_abs_q" or "all_q_in_interval"

    Returns:
    --------
    tuple (q, p, spin, formula) or list of tuples
    """
    p_n = k * n * (n + 1)
    results_q = []

    for q in range(q_min_range, q_max_range + 1):
        p1 = p_n + 1 + 2 * k * q
        if p1 > 1 and isprime(p1):
            results_q.append((q, p1, 1, "p1"))

        p2 = p_n - 1 + 2 * k * q
        if p2 > 1 and isprime(p2):
            results_q.append((q, p2, -1, "p2"))

    if mode_q == "min_abs_q":
        if results_q:
            results_q.sort(key=lambda x: (abs(x[0]), x[0]))
            return results_q[0]
        return (None, None, None, None)
    elif mode_q == "all_q_in_interval":
        return results_q
    else:
        raise ValueError(f"Unknown mode_q: {mode_q}")

# Global parameters
PARAMS = {
    "k": 21,
    "n_max": 1000,
```

```
    "q_min_range": -20,
    "q_max_range": 20,
    "mode_q": "min_abs_q"
}

# Main execution
results = []
for n in range(PARAMS["n_max"] + 1):
    result = find_prime(n, PARAMS["k"],
                        PARAMS["q_min_range"],
                        PARAMS["q_max_range"],
                        PARAMS["mode_q"])

    if PARAMS["mode_q"] == "min_abs_q":
        q, p, spin, formula = result
        if p is not None:
            num_digits = len(str(p))
            N = primepi(p)  # O(p^(2/3)) vs O(p) for naive method
            results.append((n, q, p, num_digits, spin, formula, N))
```

Listing 1: Optimized implementation for prime search

B Code for large prime search

```
import gmpy2
from gmpy2 import mpz, is_prime
import multiprocessing as mp
from functools import lru_cache

def miller_rabin_large(n, rounds=40):
    """
    Miller--Rabin test optimized for large numbers (gmpy2).

    Complexity: O(rounds * log^3 n)
    Accuracy: Error < 4^(-rounds)
    """
    if n < 2:
        return False

    # Write n-1 = 2^r * d
    r = 0
    d = n - 1
    while d % 2 == 0:
        r += 1
        d //= 2

    for _ in range(rounds):
        a = gmpy2.mpz_random(gmpy2.random_state(), n - 3) + 2
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
```

```
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

def find_large_prime(D, k=21, window=10**6, max_q=100):
    """
    Search for a prime with D decimal digits.

    Parameters:
    -----------
    D : int - Desired number of digits
    k : int - Multiplicative coefficient
    window : int - Half-window around n_target for n search
    max_q : int - Bound for q exploration

    Returns:
    --------
    tuple (p, n, q, formula) or None
    """
    n_target = int((10**(D-1) / k)**0.5)

    for n in range(n_target - window, n_target + window):
        p_base = k * n * (n + 1)
        for q in range(-max_q, max_q + 1):
            p1 = p_base + 1 + 2 * k * q
            p2 = p_base - 1 + 2 * k * q

            if p1 > 10**(D-1) and miller_rabin_large(mpz(p1), rounds=50):
                return (p1, n, q, "p1")
            if p2 > 10**(D-1) and miller_rabin_large(mpz(p2), rounds=50):
                return (p2, n, q, "p2")

    return None

def parallel_search(D, k=21, num_workers=8):
    """Parallel search over multiple windows."""
    with mp.Pool(num_workers) as pool:
        results = pool.starmap(
            find_large_prime,
            [(D, k, 10**6, 100) for _ in range(num_workers)]
        )
    return [r for r in results if r is not None]
```

Listing 2: Large prime search

C Supplementary proofs and complexities

C.1 Algorithm complexities

Proposition C.1 (Complexity of Algorithm 2)**.** The time complexity is $O(n_{\max} \cdot (q_{\max} - q_{\min}) \cdot M(p))$, where $M(p) = O(\log^3 p)$ for the Miller–Rabin test.

Proposition C.2 (Complexity for large numbers)**.** For the large prime search algorithm described in Section B with D-digit numbers:

- Per candidate: $O(D^3)$ (naive Miller–Rabin) or $O(D^2 \log D \log\log D)$ (with FFT multiplication).
- Expected number of candidates: $O(D \ln 10)$.
- Total complexity: $O(D^4 \ln D)$ or $O(D^3 \ln^2 D)$ with FFT.